# AI-103: Developing AI Apps and Agents on Azure

Comprehensive Practice Questions Compilation

## QUESTION 1: MULTI-TURN RAG ARCHITECTURE

> You are developing an autonomous Customer Support Agent using Microsoft Foundry. The agent needs to answer customer inquiries based on your company's internal product documentation while strictly preventing indirect prompt injection attacks and ensuring the model does not generate answers outside the provided data scope.
>
> You need to implement the solution using Azure AI services. Which two actions should you perform? (Each correct answer presents part of the solution. Choose two.)

**A.** Implement Prompt Shield in Azure AI Content Safety to detect malicious user inputs.

**B. Configure a Groundedness detection policy in Azure AI Content Safety to evaluate model outputs against the context dataset.** CORRECT

**C. Use Azure AI Search to build a vector and keyword hybrid index of the internal documentation for data retrieval.** CORRECT

**D.** Deploy an Azure API Management gateway with rate-limiting policies directly in front of the model endpoint.

**E.** Configure Role-Based Access Control (RBAC) using the Contributor role for user authentication to the agent.

**Explanation:** A standard Retrieval-Augmented Generation (RAG) pattern requires a robust retrieval framework like **Azure AI Search** using hybrid capabilities to cleanly pull context data. To ensure that the model stays within boundaries and doesn't hallucinate outside of this data scope, a **Groundedness detection** content safety policy must be set to validate outputs directly against the context dataset.

## QUESTION 2: MIXED LANGUAGE TRANSCRIPTS

You have an application named App1 that uses Azure Speech in Foundry Tools to transcribe live calls.

Transcript segments often contain both English and Spanish. App1 sends each segment to Azure Translator in Foundry Tools to translate to another language.

Sometimes, mixed-language segments result in incomplete or incorrect translations. You need to reduce translation errors. The solution must ensure that the entire transcript is translated successfully.

What should you do before sending the segments to Translator?

**A.** Use document translation to translate the entire transcript as a single document.

**B.** Enable automatic language detection for the translation request.

**C. Split the mixed-language segments into single-language segments and translate each segment separately.** CORRECT

**D.** Specify English as the source language in the translation request for all the segments.

**Explanation:** AI translation models encounter higher error rates when multiple languages are tightly interleaved within a single short API processing chunk. Artificially splitting mixed segments into pure single-language blocks ensures pristine context and accurate linguistic alignment at the API boundary.

## QUESTION 3: CUSTOM MODEL LIFECYCLE

> You have an Azure Speech in Foundry Tools resource that hosts a custom speech to text model deployed to a custom endpoint. An agent uses the endpoint to perform real-time speech recognition.
>
> You are approaching the expiration date of the custom speech to text model. What is the expected behavior when the model expires?

**A.** The custom model will be deleted automatically when the model expires.

**B. Speech recognition requests will fall back to the most recent base model for the same locale.** CORRECT

**C.** Speech recognition requests will continue to use the expired custom model until the model is removed manually.

**D.** Speech recognition requests will return a 4xx error until a new custom model is deployed.

**Explanation:** To prevent unexpected production runtime failures, real-time custom endpoint routes are engineered to gracefully fall back directly onto the latest standard baseline model for that locale when the custom model's structural lifecycle timeline passes.

## QUESTION 4: AGENT OBSERVABILITY

You have a Microsoft Foundry project that contains a customer support agent. The agent calls an internal knowledge API tool before generating responses.

Users report the following issues:

- Some requests take more than 15 seconds to complete.
- Some responses are incorrect, even when the knowledge API returns the expected data.

You need to inspect individual agent runs to view the ordered sequence of large language model (LLM) calls, tool invocations, and timing information. Which observability capability should you use?

**A.** safety metrics

**B.** monitoring

**C.** token usage

**D.** **tracing** CORRECT

**Explanation:** Tracing isolates single execution threads, visualizing a structural sequential cascade detailing exact execution paths, parameter passing inputs/outputs, and execution timing benchmarks for every micro-operation.

## QUESTION 5: OPENAPI CONNECTION MAPPING

You have a Microsoft Foundry project named Project1 that contains the following:

- An OpenAPI tool that calls an external API
- A project connection named Connection1 that stores the API key of the external API

When an agent calls the OpenAPI tool, the API returns a 401 unauthorized error, and traces show that the API key header is NOT being sent. You need to ensure that the OpenAPI tool automatically includes the API key from Connection1 on all requests. What should you do?

**A.** Enable identity passthrough so that the tool uses the Microsoft Entra token of the caller.

**B. Connect the tool to Connection1.** CORRECT

**C.** Add the API key header manually to the OpenAPI specification.

**D.** Configure the tool to use the default connection of Project1.

**Explanation:** Secrets inside project configurations must be explicitly mapped to their corresponding tool executors. Manually binding the specific tool configuration to Connection1 instructs the runtime engine to automatically inject the authentication token headers.

## QUESTION 6: PERSISTENT CONVERSATION HISTORY

You have a customer support agent that uses the Microsoft Foundry Agent Service. Sometimes, customers return to a session days later to continue the same support case, and the agent must resume with the full historical context. The agent must provide the following:

- Multi-turn continuity within the session
- Cross-session continuity for the same case
- Access to the full interaction history, including user messages, agent messages, tool calls, and tool outputs

You need to ensure that the agent automatically reloads the complete history on each new turn. What should you do?

**A.** Enable memory summarization on the agent definition to persist the context automatically.

**B. Create and reuse a conversation by storing the conversation's ID and supplying the ID on subsequent requests.** CORRECT

**C.** Persist only the final model response stored in the client application and prepend the response to future prompts.

**Explanation:** The Agent Service natively tracks conversations using a Conversation ID state engine. Supplying this persistent ID across disparate active client application lifecycles securely instructs the backend framework to pull and append the pristine, complete sequence history.

## QUESTION 7: OPTIMIZING GENERATION COMPLETENESS

> You have a Microsoft Foundry project that contains an agent. The agent generates summaries from retrieved policy documents. You need to improve response completeness. The solution must be implemented in the logic of the application code before responses are returned. What should you do?

**A. Add a reflection pass before the responses are returned.** CORRECT

**B.** Decrease the value of the max_tokens parameter.

**C.** Decrease the value of the temperature parameter.

**D.** Switch to Retrieval Augmented Generation (RAG).

**Explanation:** Programmatic reflection adds an algorithmic self-critique loop. The application processes the candidate response against source documents, analyzing completeness limits before executing the final outbound return step.

## QUESTION 8: WORKFLOW HUMAN-IN-THE-LOOP APPROVAL

> You have a Microsoft Foundry project that contains an agent named PaymentAgent. PaymentAgent includes a function tool that issues customer refunds by using an external API. You are creating a workflow in YAML.
>
> You need to ensure that the workflow pauses for human approval and continues with the refund step only after approval is granted. How should you complete the workflow definition?

```
steps:
  - id: propose_refund
    type: agent
    agent: PaymentAgent
  - id: approval
    type: [ ask_question ]
  - id: execute_refund
    type: agent
    agent: PaymentAgent
    condition: [ approval == "approved" ]
```

**Explanation:** The `ask_question` step pattern formally pauses active graph orchestration loops to capture external async manual inputs. Downstream nodes evaluate the state payload via expression conditions like `approval == "approved"` before triggering action boundaries.

## QUESTION 9: OPENAPI SECURITY CONFIGURATIONS

You have a Microsoft Foundry project named Project1 that contains an agent. The agent uses an OpenAPI 3.0 specification to call an external weather service. The weather service requires a key to be passed in an HTTP header. The key value is stored as a connection in Project1.

You need to ensure that the key value from the connection is included automatically whenever the OpenAPI tool is invoked. What should you configure in the OpenAPI specification?

**A.** a header parameter defined for each operation

**B.** a Bearer token security scheme

**C.** **an API key security scheme** CORRECT

**D.** an Azure Key Vault connection

**Explanation:** Defining an explicit `apiKey` security scheme inside the spec components instructs the schema engine that authorization keys must map inside headers. This metadata tells Microsoft Foundry to securely pass matching stored connection profiles.

## QUESTION 10: MULTI-TENANT SEARCH ARCHITECTURE

You are building a web app named App1 that generates responses by using a model deployed to a Microsoft Foundry project named Project1. Before sending the prompts to the model, App1 must retrieve documents by using Azure AI Search.

You need to integrate Project1 and App1. The solution must meet the following requirements:

- Multiple client applications must use the same search configuration.
- A security policy must prevent key-based authentication.
- Administrative effort must be minimized.

What should you do?

**A.** Enable a managed identity for each application and call Azure AI Search directly.

**B.** Create a custom HTTP connection in Foundry and manually configure Azure AI Search endpoints per application.

**C.** Call Azure AI Search directly from each application by using Microsoft Entra authentication.

**D. Configure an Azure AI Search connection in Project1 and reference the connection in each application.** CORRECT

**Explanation:** Centralizing endpoints into a structural native **Project Connection** manages global search index rules efficiently. This structure minimizes point-to-point environment drift across multiple target web apps while leveraging platform native identity controls to suppress static keys.

## QUESTION 11: MULTI-TURN MEMORY CONTEXT SCOPE

> You have a Microsoft Foundry project that contains an agent. You need to enable long-term memory to ensure that the agent can recall user preferences across separate conversations. Stored memories must be isolated per authenticated user without the client application manually generating user IDs.
>
> How should you complete the Python code?

```
memory_tool = MemorySearchTool(
    memory_store_name=mem_store_name,
    scope="{{$userId}}"
)
agent_def = PromptAgentDefinition(
    model="gpt-5.2",
    tools=[memory_tool]
)
```

**Explanation:** Setting the memory container scope explicitly to the template macro variable string `"{{$userId}}"` forces automated runtime security contexts to sandbox data vectors per unique principal across disparate threads.

## QUESTION 12: POWER FX WORKSPACE FLOW RULES

> You have a Microsoft Foundry project that contains a workflow for a customer support triage process. You have an Ask a question node that stores user responses in a local variable named Var01.
>
> You need to create Power Fx expressions for an if/else condition expression that ensures that Var01 contains a value, and a Send message expression that returns the stored user response in uppercase.

- If/else condition expression: **Not(IsBlank(Local.Var01))**
- Send message expression: **Upper(Local.Var01)**

**Explanation:** In Power Fx script expressions, verifying that data is present requires executing the `Not(IsBlank(...))` configuration hierarchy. String manipulations utilize structural operations like `Upper(...)` applied on explicitly scoped parameters.

## QUESTION 13: INDEXING OCR DOCUMENT PROCESSING

You have a Microsoft Foundry project that contains an agent. The agent uses Azure AI Search as the retriever. You plan to ingest PDFs into an Azure AI Search index to ensure that the agent can ground responses in texts in both documents and embedded images. Users require citations that link to the source files.

You need to ensure that during indexing, the images are extracted into a structure that can be used as input for the built-in optical character recognition (OCR) skill. Which indexing approach should you use?

**A.** the outputFieldMappings parameter to write image data to a searchable field

**B.** a skillset to run the OCR skill directly against the content field of the index

**C.** a Shaper skill to restructure the OCR input

**D.** **an indexer to extract image data into a normalized_images collection** CORRECT

**Explanation:** Binary document cracking configurations inside an indexer require initializing specific configuration targets like `generateNormalizedImages` to explicitly convert graphics fragments into a manageable, structured `normalized_images` metadata collection array.

## QUESTION 14: STRUCTURED DOCUMENT EXTRACTION ANALYSIS

You have a Microsoft Foundry project that ingests scanned PDF invoices stored in Azure Blob Storage. Each invoice contains printed line items and has a table-based layout. Extracted results are stored as structured JSON and used as grounding data for an agent in a Retrieval Augmented Generation (RAG) solution.

You need to create a single analyzer that extracts specific invoice fields across varying templates and returns confidence scores so that results with confidence below 0.80 can be routed for supervisor review. What should you use?

**A.** a Foundry agent that has groundedness guardrails enabled to extract invoice fields and confidence scores

**B.** **a custom Azure Content Understanding in Foundry Tools analyzer that defines the required fields as the extracted fields and the returned confidence scores for routing** CORRECT

**C.** the Azure Content Understanding in Foundry Tools prebuilt-layout analyzer

**D.** the Azure Content Understanding in Foundry Tools prebuilt-documentSearch analyzer and search.score from the Azure AI Search results for routing

**Explanation:** Custom schema definitions inside Azure Content Understanding capture targeted key-value items seamlessly across dynamic formats while returning distinct mathematical data-extraction confidence values per parsed object field.

## QUESTION 15: OPERATIONAL RUNTIME EVALUATION TRACKING

> You have a Microsoft Foundry project that contains an agent. You need to enable evaluation of the agent using a chat conversation. The solution must minimize the effort required to collect data and run evaluations automatically. Which type of evaluation should you use?

**A. continuous** CORRECT

**B.** manual

**C.** batch

**D.** policy

**Explanation:** Continuous evaluation automatically pulls active production telemetry logs from live conversation streams, running automated background processes to compute system quality metrics with zero intervention or compilation overhead.